
SRD Arena

Projektbericht

Arbeitsfassung

Einführung in Python

Heinrich-Heine-Universität Düsseldorf

1. September 2026

Inhaltsverzeichnis

1	Einleitung	1
1.1	Motivation	1
1.2	Zielsetzung und Umfang	1
2	Bibliotheken und Entwicklungswerkzeuge	2
3	Konzept und Implementierung	3
3.1	Programmstruktur	3
3.2	Spieldaten als JSON	4
3.3	Datenimport und Validierung (content)	4
3.4	Modellierung der Regellogik (domain)	5
3.5	Anwendungssteuerung und Schnittstellen (engine)	5
3.6	Technischer Ablauf am Beispiel eines Zaubers	5
3.7	KI-Unterstützung im Entwicklungsprozess	6
4	Ergebnisse	9
4.1	Funktionsnachweis	9
4.2	Beispieldurchlauf eines Encounters	9
5	Diskussion und Fazit	10
5.1	Zielerreichung	10
5.2	Abweichungen vom Plan	10
5.3	Kritische Würdigung	10
5.4	Ausblick	11
6	Literatur- und Quellenverzeichnis	12
7	Anhang	12
7.1	Bedienungsanleitung	12
7.2	Ausgewählte Code-Snippets	12

1 Einleitung

Dungeons and Dragons ist ein beliebtes Pen and Paper-Rollenspiel, in dem taktische, rundenbasierte Kämpfe einen großen Teil des Spiels ausmachen. Üblicherweise sind die Teilnehmer eines Kampfes ein Team von Spielern und ein Team von Gegnern, die vom Spielleiter gesteuert werden. Das System Reference Document 5.2.1 (SRD) ist eine unter der Creative Commons License CC-BY-4.0 verfügbare Fassung der Spielregeln, die den größten Teil der Grundmechaniken des Spiels enthält, in der aber viele Inhalte des eigentlichen Spiels fehlen. So kann ein Spieler beispielsweise für seinen Charakter eine Klasse und Subklasse wählen, jedoch steht pro Klasse nur eine Subklasse zur Verfügung, während im proprietär lizenzierten Spielerhandbuch pro Klasse vier Subklassen zur Auswahl stehen. Dieses Programm, SRD Arena, setzt zunächst einen Teil des Kampfsystems des SRD als rundenbasiertes 2D-Spiel um. Es gibt mehrere vordefinierte Kampfszenarien, die die umgesetzten Funktionalitäten demonstrieren. Anhand von Vorlagen können in weiteren Dateien auch weitere Szenarien vom Nutzer definiert und anschließend gespielt werden.

1.1 Motivation

Die meisten Spielleiter entwerfen die Kampfszenarien für ihre Spieler bereits vor dem eigentlichen Spieltermin. Aus Sicht des Spielleiters gehört zu den Zielen eines Kampfes, den Spielern eine taktische Herausforderung zu bieten, ohne dass der Kampf zu schwer wird.

Im Rahmen der umgesetzten Regeln können Spielleiter mit diesem Programm ihre Kampfszenarien vor dem eigentlichen Spiel mehrfach ausprobieren. Das ist bereits vorteilhaft, weil im Programm viele Berechnungen und Würfe von Würfeln automatisch stattfinden, die sonst händisch geschehen müssten. Mittelfristig, aber nach Abgabe des Projektes, soll der gesamte Umfang der SRD-Regeln abgedeckt werden.

Langfristig wäre es auch möglich, das Projekt um eine Option zu erweitern, die Kampfentscheidungen aller Teilnehmer durch klügere Bots automatisiert. Dann könnten Kämpfe vielfach simuliert werden, um Statistiken zu erzeugen. Für einen Spielleiter wäre dann nicht nur erkennbar, ob ein Encounter einmal gewonnen wurde, sondern auch, wie häufig ein Team gewinnt und welche Teilnehmer besonders großen Einfluss auf das Ergebnis haben. Die im Projekt umgesetzte Kampfbengine soll dafür später die technische Grundlage bilden.

1.2 Zielsetzung und Umfang

SRD Arena ist eine Engine für taktische Kämpfe auf einem 2D-Spielfeld nach dem SRD 5.2.1-Regelwerk. In einer GUI kann der Nutzer Kampfszenarien spielen. Die Steuerung der Teilnehmer erfolgt wahlweise durch Nutzerinput, oder durch sehr simple Bots. In strukturierten Textdateien können Kampfszenarien definiert werden. Ein Kampf besteht aus Teams, die Mitglieder beinhalten. Jedes Mitglied hat eine Startposition auf dem Spielfeld, sowie eine Referenz auf eine Definition ihrer Attribute und Fähigkeiten.

Teilnehmer

Während des Kampfes verwaltet die Engine für jeden Teilnehmer unter anderem Lebenspunkte, Position, Bedingungen und verbleibende Aktionsressourcen. Werte wie Rüstungsklasse, Bewegungsgeschwindigkeit, Angriffe und bekannte Zauber stammen aus der zugehörigen Kreaturendefinition.

Rundenablauf

Zu Beginn des Encounters bestimmt ein Initiativewurf die Reihenfolge, in der Teilnehmer ihre Züge ausführen. Nach dem Zug des letzten Teilnehmers beginnt eine neue Runde. Während seines Zuges verfügt ein Teilnehmer grundsätzlich über Bewegung, eine Aktion und eine Bonusaktion. Die Engine verwaltet diese Ressourcen und verhindert ihre mehrfache Verwendung. Bestimmte Ereignisse können eine Reaktion eines anderen Teilnehmers auslösen und den laufenden Zug vorübergehend unterbrechen. Eine implementierte Anwendung ist der Gelegenheitsangriff beim Verlassen der Reichweite eines Gegners. Eine verbrauchte Reaktion wird zu Beginn des nächsten eigenen Zuges wieder verfügbar.

Das Spielfeld

Die Engine modelliert das Spielfeld als Gitter aus quadratischen Feldern. Die Seite eines Feldes repräsentiert den Regeln entsprechend fünf Fuß in der Spielwelt.

Begrenzung des Umfangs

Der Umfang der Abgabe ist bewusst kleiner als der Umfang des SRD. Unterstützt wird jeweils ein einzelner Encounter mit eingeschränkten Kampfregeln. Nicht Teil der Abgabe sind eine vollständige Abbildung aller Klassen und Zauber, eine allgemeine Ausrüstungsverwaltung sowie mehrere aufeinanderfolgende Encounter. Diese Begrenzung ermöglicht es, die unterstützten Regeln innerhalb eines vollständigen Kampfablaufs zu verbinden und zu testen.

2 Bibliotheken und Entwicklungswerkzeuge

Zur Laufzeit verwendet SRD Arena hauptsächlich Pydantic und PySide6. Pydantic bildet die Schemata der JSON-Inhalte ab und validiert eingelesene Daten, bevor daraus Objekte des Fachmodells entstehen. Dadurch müssen Struktur- und Typprüfungen nicht für jedes Eingabeformat manuell implementiert werden. Die Desktop-GUI ist mit PySide6 implementiert. Eine Alternative wäre eine browserbasierte Oberfläche gewesen, aber zum Zeitpunkt der Entscheidung war ich aus den Übungen noch mit PySide6 vertrauter. Eine browserbasierte Nutzeroberfläche hätte bereits Grundlagen dafür gelegt, SRD Arena irgendwann online bereitzustellen.

Für automatisierte Tests wird pytest genutzt. Hypothesis ergänzt beispielbasierte Tests um generierte Eingaben, mit denen unter anderem die Grenzen der Lebenspunktlogik geprüft

werden. `pytest-cov` misst die erreichte Testabdeckung, `mypy` prüft die Typannotationen, `Ruff` prüft die einheitliche Formatierung, `Interrogate` misst die Abdeckung durch Dokumentation.

Während der Entwicklung wurden mit GitHub Actions durch eine CI-Pipeline die `mypy`- und `ruff`-Prüfungen laufend durchgeführt. Die CI-Pipeline erzeugt auch mithilfe von Sphinx eine GitHub Pages-Dokumentationsseite.

Die Python-Abhängigkeiten sind in der `pyproject.toml` erfasst und werden mit `uv` installiert. `uv` wird außerdem verwendet, um die Entwicklungswerkzeuge in einer einheitlichen Projektumgebung auszuführen.

`Pydantic` und `PySide6` werden zur Laufzeit von der Anwendung benötigt. Die übrigen Pakete dienen nur der Entwicklung und Dokumentation.

3 Konzept und Implementierung

3.1 Programmstruktur

Der Quellcode ist nach Verantwortungsbereichen in die Python-Pakete `content`, `domain`, `engine` und `frontends` gegliedert. Neben dem Quellcode gibt es ein Content-Verzeichnis, in dem Spielregeln als JSON-Dateien hinterlegt sind. Das Content-Paket liest die in JSON-Dateien definierten Spieldaten, validiert sie und übersetzt sie in Domain-Objekte. Die Domain enthält den veränderlichen Encounterzustand sowie die davon unabhängigen Kampfregeln. Die Engine stellt mit `Session`, `Commands` und `Observations` eine frontendneutrale Schnittstelle bereit und koordiniert die Interaktion mit der Domain. Die Frontends setzen Benutzereingaben in Engine-Befehle um und stellen die zurückgegebenen Beobachtungen dar. Die Abhängigkeiten verlaufen überwiegend in Richtung der Domain: Diese kennt weder Content, Engine noch Frontends. Die Engine verwendet die Domain, ist aber nicht für das Laden von Dateien zuständig. Die Frontends steuern einen laufenden Kampf über die Engine-Schnittstelle. Direkte Zugriffe auf den Content-Bereich dienen lediglich der Encounter-Auswahl und Darstellungskonfiguration; die GUI verwendet zusätzlich zustandslose Geometriefunktionen für visuelle Vorschauen. Architekturtests prüfen diese Paketgrenzen. Die folgenden Abschnitte erläutern Datenverarbeitung, Regellogik und Anwendungssteuerung getrennt, bevor Abschnitt 3.6 ihr Zusammenspiel am Beispiel des Spellcastings zeigt.

Die Aufteilung richtet sich damit nach Verantwortlichkeiten. Innerhalb der einzelnen Pakete gibt es eine weitere Aufteilung nach Spielfunktionen. Ein Zauber durchläuft zum Beispiel mehrere Bereiche, ohne dass das Einlesen seiner JSON-Datei, seine Regelwirkung und seine Darstellung in derselben Klasse behandelt werden.

[Bild]

3.2 Spieldaten als JSON

Das SRD ist eine PDF-Datei, in der alle Regeln in Prosa stehen. Es enthält zahlreiche Zauber, die sich in ihrer Funktionsweise ähneln. Betrachten wir zum Beispiel Cone of Cold und Burning Hands:

```
Cone of Cold:
You unleash a blast of cold air. Each creature in a 60-foot Cone originating from you makes a
↪ Constitution saving throw, taking 8d8 Cold damage on a failed save or half as much damage on a
↪ successful one. [...]

Burning Hands:
A thin sheet of flames shoots forth from you. Each creature in a 15-foot Cone makes a Dexterity saving
↪ throw, taking 3d6 Fire damage on a failed save or half as much damage on a successful one.
```

Die Zauber verwenden unterschiedliche Schadenswürfel und Schadensarten, wirken aber beide in einem Kegel und verlangen einen Rettungswurf. Lediglich die Größe des Kegels und die konkreten Werte unterscheiden sich. Hier zeichnen sich bereits Mechanismen ab, die wiederverwendet werden können.

Die Seite 5e.tools, die unter der MIT-Lizenz ihren Quellcode verfügbar macht, hat bereits alle Zauber in ihr eigenes JSON-Schema überführt. Dieses Schema war nicht perfekt für SRD Arena geeignet, denn auch darin waren einzelne Mechanismen noch in Prosa versteckt. Es lieferte trotzdem bereits eine bessere Grundlage zur Weiterarbeit als die originale PDF-Datei, da das Schema nur umgeformt und erweitert werden musste und Inhalte auch für KI besser lesbar waren.

Für das Projekt wurde für die implementierten Zauber ein erweitertes JSON-Schema entworfen, sodass die zugrundeliegenden Mechaniken nur einmal implementiert und dann von den jeweiligen Zaubern verwendet werden können. Für die Beispiele Cone of Cold und Burning Hands befinden sich im Anhang die Daten in JSON-Form, wie sie vom Programm verwendet werden.

Außerdem werden auch nutzerdefinierte Kampfszenarien direkt als JSON-Dateien geschrieben.

3.3 Datenimport und Validierung (content)

Die JSON-Inhaltsdateien werden durch die Content-Ebene eingelesen und mit Pydantic-Schemata validiert. Bei einem Kampfszenario werden unter anderem positive Spielfeldmaße, eindeutige Teilnehmerkennungen, gültige Teamzuordnungen und unterschiedliche Startpositionen je Teilnehmer auf dem Spielfeld geprüft. Anschließend übersetzen Builder die validierten Daten in Objekte der Domain. Fehlerhafte Inhalte werden bereits hier abgelehnt, damit die übrigen Bereiche mit bereits geprüften Inhalten arbeiten können.

3.4 Modellierung der Regellogik (domain)

Die Domain bildet die für den Kampf relevanten Konzepte wie Aktionen, Zauber und Effekte ab. Neben deren Definitionen enthält sie den veränderlichen Zustand eines laufenden Kampfes sowie die Regeln für Bewegung, Angriffe, Rettungswürfe, Schaden, Heilung und Bedingungen. Diese Regeln bestimmen, welche Aktionen im aktuellen Kamp fzustand zulässig sind und wie ihre Ausführung den Zustand verändert. Der wichtige Unterschied zur Content-Ebene ist, dass im Content die Form der Eingabedateien modelliert wird, sodass diese in die Python-Objekte der Domain-Ebene übersetzt werden können. Die Regeln für Bewegung, Angriffe, Rettungswürfe, Schaden, Heilung und Bedingungen arbeiten auf diesem Zustand, kennen aber weder JSON-Dateien noch Elemente der GUI.

3.5 Anwendungssteuerung und Schnittstellen (engine)

Die Engine bildet die Schnittstelle zwischen Domain und Frontends. Sie gibt den aktuellen Kamp fzustand und die zulässigen Aktionen als frontendneutrale Observation auf. Ein Frontend übermittelt die Auswahl des Nutzers als Befehl an die Engine, die dessen Ausführung an die Domain weitergibt und anschließend eine aktualisierte Observation erzeugt. Dadurch verwenden die GUI und das Headless-Frontend dieselbe Schnittstelle und müssen die interne Struktur der Domain nicht kennen.

3.6 Technischer Ablauf am Beispiel eines Zaubers

Beim Laden eines Encounters liest die Content-Ebene die Zauberdateien ein, validiert sie mit dem Pydantic-Modell `SpellSchema` und legt sie in einem Katalog ab. In Domänenobjekte vom Typ `Spell` werden anschließend nur die Zauber übersetzt, auf die eine Kreatur des Encounters verweist. Bei `Burning Hands` überführt der Builder insbesondere das JSON-Feld `capability` in eine ausführbare Definition für Zielgebiet, Rettungswurf, Schaden und Skalierung. Das erzeugte Objekt wird dem Spellcasting der Kreatur zugeordnet; während des Kampfes muss die JSON-Datei daher nicht erneut ausgewertet werden.

Ist die Kreatur am Zug, leitet die Domain aus ihren bekannten Zaubern die derzeit zulässigen `EncounterAction`-Objekte ab. Für `Burning Hands` entstehen dabei auch Varianten für höhere verfügbare Zauberplätze. Die Engine nimmt diese Aktionen in die Observation für das Frontend auf. Nach der Auswahl legt der Nutzer auf dem Spielfeld die Richtung des 15 Fuß langen Kegels fest; das Frontend sendet diese Konfiguration über die Engine zurück.

Die konfigurierte Aktion gelangt über die Engine zum `EncounterOrchestrator` der Domain. Bei der Ausführung werden die Aktionsressource und der gewählte Zauberplatz verbraucht und anschließend die `Capability` aufgelöst. Die Domain bestimmt alle Kreaturen im Kegel und führt für sie Geschicklichkeitsrettungswürfe aus. Bei einem Fehlschlag verursacht der Zauber 3d6 Feuerschaden, bei einem Erfolg die Hälfte. Für jede Stufe des Zauberplatzes oberhalb der ersten kommt 1d6 hinzu. Der Schaden wird auf den `EncounterState` angewendet; aus dem neuen Zustand sowie den erzeugten Meldungen und Ereignissen erstellt die Engine die nächste

Observation. Das im Regeltext ebenfalls beschriebene Entzünden brennbarer Gegenstände ist in der Inhaltsdatei ausdrücklich als noch nicht umgesetzt gekennzeichnet.

Das Capability-Schema zerlegt Zauber in wiederverwendbare Bausteine für Zielauswahl, Auflösung, Effekte und Skalierung. `Burning Hands` und `Cone of Cold` verwenden dieselbe Struktur: Beide wirken als vom Zaubernden ausgehende Kegel, verlangen einen Rettungswurf, verursachen Flächenschaden und halbieren diesen bei einem erfolgreichen Wurf. Unterschiede wie Kegellänge, Rettungswurfsattribut, Schadensart, Schadenswürfel und Skalierung stehen in den jeweiligen Inhaltsdaten. `Hold Person` gehört zu einer anderen Zauberfamilie: Statt Flächenschaden betrifft der Zauber einzelne Humanoide und erzeugt bei einem fehlgeschlagenen Rettungswurf den Zustand `Paralyzed`, der durch wiederholte Rettungswürfe beendet werden kann. Trotzdem verwendet er dieselben allgemeinen Bausteine für Zielauswahl, Rettungswurf und Effekt. Dadurch benötigen diese Zauber keinen jeweils vollständig eigenen Ausführungspfad.

3.7 KI-Unterstützung im Entwicklungsprozess

Einsatzumfang und Arbeitsteilung

In diesem Projekt wurde der überwiegende Teil der Programmlogik in vielen Iterationen mit KI generiert. Meist formulierte ich eine möglichst genaue Anforderung für ein Feature oder einen Bug, ließ Codex einen Änderungsvorschlag erstellen und prüfte anschließend den generierten Code und die Auswirkungen auf das fertige Programm.

Viele Ideen zur Architektur wurden auch mit KI-Unterstützung auf Lücken geprüft und angepasst. Außerdem wurden Regelabschnitte, die in den ursprünglichen JSON-Dateien noch in Prosaform gegeben waren, mithilfe von KI in das entworfene JSON-Schema übersetzt. In Stichproben sind dabei keine Fehler aufgefallen, aber für eine finale Version sollte die Korrektheit jedes Zaubers manuell sichergestellt werden.

KI-generierte Bestandteile

Vorgabe: Aufführen, welche Module, Funktionen oder Tests maßgeblich mit KI-Unterstützung, beispielsweise durch ChatGPT oder Copilot, erstellt wurden. Außerdem begründen, weshalb sich der KI-Einsatz dafür eignete, etwa bei Boilerplate-Code, Standardalgorithmen oder regulären Ausdrücken.

- Übersetzung von verbleibender Regelprosa in JSON (Spells)
- Schrittweise Implementierung von Regeln - Hit Points, Angriffe, Spells, etc.

KI-Unterstützung wurde vor allem für wiederkehrende Implementierungsschritte genutzt. Dazu gehörten ähnliche Schemafelder und Builder, Tests nach bereits vorhandenen Mustern sowie die Übertragung ausgewählter Zauberregeln in das Capability-Schema. Auch bei der schrittweisen Ergänzung von Lebenspunkten, Angriffen und Zaubern erzeugte Codex große Teile des ersten Entwurfs. Das war besonders dann nützlich, wenn ein im Projekt bereits verwendetes Muster auf weitere Inhalte übertragen werden sollte.

Selbst programmierte Bestandteile

Vollständig selbst geschriebene größere Module gibt es nicht. Nicht an Codex delegiert werden konnten jedoch die fachliche Auswahl der umgesetzten Regeln und die abschließende Bewertung, ob eine Lösung zum Projekt passt. Dazu gehörten insbesondere das Streichen nicht mehr benötigter Features, die Entscheidung für einen einzelnen Encounter und die Kontrolle von Grenzfällen anhand des SRD-Regeltexts. Diese Arbeit führte häufig dazu, dass generierter Code anschließend umgebaut oder wieder entfernt wurde.

Einfluss auf Architektur- und Designentscheidungen

Vorgabe: Darstellen, ob die KI bereits bei der Planung und Architektur des Projekts konsultiert wurde. Falls dies der Fall war, erläutern, welche Vorschläge übernommen wurden und an welchen Stellen bewusst von den Empfehlungen der KI abgewichen wurde, weil die eigene Einschätzung besser zum Projekt passte.

- Lernen über Layered Architecture
 - Schemadesign für Laden von Content
 - Lösungsvorschläge waren meist nach Iteration gut, aber das Erkennen der Problematik lag oft bei mir
1. Früheres Refactoring: Bei Implementierung von Zaubern wurde FFahrplan in Zusammenarbeit mit KI aufgestellt, aber der Code wurde schnell sehr unübersichtlich. Auf Nachfrage schlug Codex vor, dem Fahrplan vor einem Refactoring noch relativ weit zu folgen, aber ich entschloss, das Refactoring früher zu beginnen, um auch früher einen Zustand zu erreichen, in dem ich den Code leichter nachvollziehen und selbst anpassen kann.
 2. SRD Arena hatte in der experimentellen Phase noch einen Spieler und einen Gegner. Daraus ist gewachsen, dass es eine primäre Kreatur auf dem Spielbrett gibt. Viele weitere Strukturen wurden dann parallel einmal für "den Spieler", dann für seine Verbündeten und seine Gegner geschrieben, obwohl die Logik eigentlich einheitlich sein sollte.
 3. Recherche für Regelparsing in Grenzfällen, Beispiel Calm Emotions und Immunitäten.
 4. Conditions:
 - Unconscious implies Incapacitated and causes the creature to fall Prone.
 - When Unconscious ends, Incapacitated ends with it.
 - Prone can remain independently. -> Domänenwissen verhindert Abstraktion, die KI vornehmen wollte
 5. Funktionsweise von Spells aus tagged Prosa extrahieren vs. JSON enrichen:
 - Weniger Programmcode zum Parsen von Authored content
 - Teile der Definition werden nicht in Logik versteckt
 6. Beginnen Dodge, Disengage, etc. in Domain oder in content?

Die Vorschläge der KI wurden nicht unverändert übernommen. Beispielsweise empfahl die KI zunächst, vor der Umstrukturierung des Content-Pakets weitere repräsentative Zauber zu implementieren. Diese Reihenfolge wurde bewusst verworfen, da dadurch zusätzliche Funktionalität auf einer bereits als unübersichtlich erkannten Struktur aufgebaut worden wäre. Ebenso wurde eine erste Implementierung generischer Kreaturentyp-Anforderungen nicht dauerhaft in dieser Form beibehalten, sondern später in das breitere Capability-Schema überführt. Zunächst wurden die Zauberdaten um explizite ausführbare Mechaniken erweitert, um die spätere Implementierung auf einer konsistenten Datenbasis aufzubauen. Bei der Modellierung von Zuständen wurden Vorschläge der KI außerdem anhand des Regeltexts überprüft und korrigiert, insbesondere hinsichtlich der Unterscheidung zwischen Immunität und Unterdrückung einer Bedingung.

Die KI war besonders hilfreich, um mögliche Aufteilungen und Abhängigkeiten sichtbar zu machen. Die Richtung einer Umstrukturierung musste trotzdem aus dem Gesamtzustand des Projekts abgeleitet werden. Ein Beispiel ist die Trennung von Content, Domain und Engine: Codex half beim Verschieben einzelner Verantwortlichkeiten, während ich darauf achten musste, dass keine neuen Rückabhängigkeiten entstehen. Ein weiteres Beispiel sind voneinander abhängige Conditions. Eine allgemein wirkende Abstraktion war dort nicht automatisch regelkonform, weil etwa Unconscious zwar Incapacitated verursacht, Prone nach dem Ende von Unconscious aber unabhängig bestehen bleiben kann.

Codegenerierung, Prüfung und Iteration

Vorgabe: Die Zusammenarbeit mit der KI beschreiben. Dabei insbesondere erläutern, ob generierter Code direkt übernommen oder in kritischen Iterationsschleifen geprüft und überarbeitet wurde. Dazu gehören auch manuelle Korrekturen von KI-Fehlern wie Halluzinationen oder veralteter Syntax.

Die Zusammenarbeit erfolgte überwiegend in kurzen Iterationen. Nach einer Beschreibung des gewünschten Verhaltens untersuchte Codex die betroffenen Dateien und erstellte einen Änderungsvorschlag. Danach wurden Tests, mypy und Ruff ausgeführt. Bei Fehlern oder einer unpassenden Struktur wurde nicht nur die konkrete Fehlermeldung behoben, sondern erneut geprüft, ob die gewählte Verantwortung im richtigen Paket liegt. Größere Änderungen wurden deshalb häufig durch weitere Aufräumschritte und Architekturtests abgesichert.

Generierter Code wurde nicht allein deshalb übernommen, weil die Tests erfolgreich waren. Mehrfach waren Lösungen lokal funktionsfähig, führten aber neue Sonderfälle oder parallele Modelle für denselben Sachverhalt ein. Solche Probleme traten besonders bei älteren Strukturen für Szenen, Ausrüstung und Spielercharaktere auf. Die Iteration bestand dann darin, den noch benötigten Anwendungsfall enger zu formulieren, überflüssige Teile zu entfernen und die verbleibende Lösung erneut gegen Tests und Regeltext zu prüfen.

4 Ergebnisse

4.1 Funktionsnachweis

Die umgesetzte Funktionalität lässt sich anhand der mitgelieferten Showcase-Encounter nachvollziehen. Sie decken unter anderem normale Angriffe und Multiattacks, Bewegung und Gelegenheitsangriffe, unmittelbaren Zauberschaden, Zustände, anhaltende Zaubereffekte, Heilung mit einer auf mehrere Ziele verteilten Ressource sowie einzelne Klassenfähigkeiten ab. Abbildung [X] zeigt die GUI während eines Encounters; die angebotenen Schaltflächen entsprechen dabei den Aktionen, die die Engine im aktuellen Zustand als zulässig meldet.

Zusätzlich werden die Regeln und Paketgrenzen durch automatisierte Tests geprüft. Zum Zeitpunkt der Abgabe liefen [Anzahl] Tests einschließlich der Doctests ohne Fehler durch. `mypy --strict`, `ruff check` und `ruff format --check` meldeten [Ergebnis ergänzen]. Interrogate erreichte eine Dokumentationsabdeckung von [Wert ergänzen]. Diese Ausgaben sollten als kurze Tabelle oder als Ausschnitt aus der Konsole dargestellt werden, weil sie die formalen Qualitätskriterien direkt belegen.

4.2 Beispieldurchlauf eines Encounters

Nach dem Start mit `uv run srd-arena` zeigt das Programm zunächst die verfügbaren Encounter an. Für diesen Beispieldurchlauf wird „Immediate Damage Spells“ gewählt. Der Encounter enthält den Zaubersyrker Spectrum Adept sowie mehrere gegnerische Ziele auf einem 18 mal 12 Felder großen Spielfeld. Beim Start erzeugt die Engine den veränderlichen Encounterzustand aus den geladenen Definitionen und würfelt die Initiative.

Sobald Spectrum Adept an der Reihe ist, zeigt die GUI seine möglichen Bewegungen und Aktionen. Der Nutzer wählt `Burning Hands` und legt anschließend die Richtung des Kegels auf dem Spielfeld fest. Eine Vorschau markiert den Wirkungsbereich. Nach der Bestätigung bestimmt die Domain die betroffenen Ziele, führt für jedes Ziel einen Geschicklichkeitsrettungswurf aus, zieht den ausgewürfelten oder halbierten Schaden ab und verbraucht den verwendeten Zauberplatz. Die Ergebnisse erscheinen sowohl an den Kreaturen als auch im Kampfprotokoll. Abbildung [X] zeigt die Wahl des Zielgebiets, Abbildung [Y] den Zustand nach der Auflösung.

Danach geht die Initiative zum nächsten Teilnehmer über. Da in diesem Showcase alle Teilnehmer extern gesteuert werden, kann der Nutzer die Züge der übrigen Kreaturen ohne weitere Aktion beenden. Der Ablauf aus Entscheidung, Auflösung und aktualisierter Darstellung wiederholt sich, bis nur noch ein Team kampffähige Teilnehmer besitzt. Anschließend meldet die Engine das Ergebnis und bietet einen Neustart oder das Beenden der Anwendung an.

5 Diskussion und Fazit

5.1 Zielerreichung

Das für die Abgabe eingegrenzte Ziel wurde erreicht. Encounter können aus JSON-Dateien geladen, validiert, in der GUI ausgewählt und bis zu ihrem Ende gespielt werden. Die Engine verwaltet Initiative, Züge, Bewegung und Aktionsressourcen und unterstützt die im Projekt dokumentierten Angriffe, Rettungswürfe, Zauber, Bedingungen und Reaktionen. Externe Eingaben und einfache automatisch gesteuerte Teilnehmer verwenden dabei dieselbe Engine-Schnittstelle.

5.2 Abweichungen vom Plan

Der ursprünglich stärkere Schwerpunkt auf Spielercharakteren wurde verändert. Für die Abgabe erwiesen sich Monster-Statblocks und die allgemeinen Kampfregeln als bessere Grundlage, weil damit mehr unterschiedliche Encounter erstellt werden konnten, ohne zunächst eine vollständige Klassenprogression zu modellieren. Gleichzeitig erforderte die große Vielfalt der Zauber eine Entscheidung zwischen vielen einzelnen Sonderimplementierungen und einem gemeinsamen Capability-Schema. Der Aufbau dieses Schemas nahm mehr Zeit ein, verringerte danach aber die Menge an zauberspezifischem Programmcode.

5.3 Kritische Würdigung

Die Trennung zwischen geladenen Inhaltsdaten, Regellogik und Anwendungssteuerung hat mittlerweile einen guten Stand erreicht. Das entworfene Schema kann für zukünftige Funktionalitäten gut erweitert werden und die Aufgaben der einzelnen Pakete sind klar getrennt. Die automatisierten Tests waren besonders während größerer Refactorings nützlich, weil sie unbeabsichtigte Änderungen an bereits unterstützten Regeln sichtbar machten.

KI-Zusammenarbeit

Ich glaube, dass mir erst die Unterstützung durch KI ermöglicht hat, in der investierten Bearbeitungszeit ein so umfangreiches Ergebnis zu produzieren. Ich wusste bereits vor Beginn des Projektes, dass es Muster im Software-Design gibt, die sich für mein Projekt anbieten, aber ich konnte sie nicht benennen. Der Prozess, über diese zu recherchieren und den genauen Bezug zu meiner Anforderung herzustellen, war aber mit Hilfe eines Chatbots deutlich schneller. Mithilfe eines Agenten war es dann auch schneller möglich, diese Muster auszuprobieren und zu versuchen, sie am Beispiel meines Projektes nachzuvollziehen. Auch die Implementierung neuer Features war zuerst weniger mühsam. Oft erreichte ich einen spielbaren Zustand für ein gegebenes Feature in wenigen Iterationen.

Ein Problem war, dass ich oft in längeren Sitzungen dazu geneigt habe, den produzierten Code irgendwann nicht mehr kritisch genug zu hinterfragen. Dann ist später aufgefallen, dass Muster

gewachsen sind, die ich sonst nicht erlaubt hätte. Das möchte ich an einem frühen Beispiel aus der Entwicklung zeigen:

In seiner ursprünglichen Fassung war SRD Arena noch als Kampfmodul für die CYOA-Engine aus den Übungen gedacht. So wären in dieser Version Kämpfe eine komplizierte Variante von Szenen, für die ein Spielfeld erzeugt wird. Nach dem Kampf würde man dann je nach Erfolg in die nächste Szene weitergeleitet werden.

Dieses Projekt hat sich auch aus meinem Code aus den Übungen weiterentwickelt. In der CYOA-Engine war der Spieler die zentrale Figur des Programmes, dem wir über die Übungen hinweg Attribute wie Lebenspunkte und ein Inventar gegeben haben, die sich dann für Kämpfe gut verwenden ließen. Für die ersten Versuche mit Kämpfen habe ich dann Gegner hinzufügen lassen, die diese Systeme auch nutzen sollten. Danach ließ ich auch das Spielerteam erweitern.

Erst später fiel mir auf, dass für Spieler, Gegner und Verbündete parallele Strukturen im Code gewachsen waren, die nicht wünschenswert waren. Ein korrekt abstrahierter Kampfteilnehmer sollte zu einem Team gehören und unabhängig davon je nach Konfiguration vom Nutzer oder von einem Bot gesteuert werden. Stattdessen war eine Mischform implementiert, an der an einigen Stellen abstrahiert wurde, aber an anderen Stellen speziell zwischen Spielern, Gegnern und Verbündeten unterschieden wurde.

Obwohl das ein frühes Beispiel war, passiert mir das Gleiche immer noch. Ich frage mich, ob die Zeitersparnis in der Implementierung die Aufräumarbeit überhaupt aufwiegt. Und selbst wenn das der Fall ist, ist dieses Muster vermutlich ein Zeichen dafür, dass ich einen zu großen Teil der Arbeit abgebe, die mir eigentlich immer Spaß gemacht hat und mir damit Arbeit schaffe, die mir weniger Freude bereitet. Trotzdem wirkt in dem Moment, in dem ich anfangen zu arbeiten, der KI-Agent wie der PPath of least resistance und ich entscheide mich, ihn weiter zu nutzen. Das Projekt war für mich das erste Mal, dass ich einen KI-Agenten in diesem Ausmaß benutzt habe und hat mir klar gemacht, dass ich weiter daran arbeiten muss, das für mich richtige Maß an KI-Unterstützung in der Entwicklung zu finden.

5.4 Ausblick

Die nächsten Meilensteine sind weitere Bestandteile der Regeln, beginnend mit der Arbeit an weiteren Zaubern. Danach wird die Unterstützung für Spielercharaktere mit Ausrüstung, Klassen, Subklassen, Charakterherkunft und Spezies erweitert.

Das Programm könnte auch von einem Szenario-Editor profitieren, damit Spielleiter ihre Kämpfe nicht in JSON-Dateien definieren müssten. Hier könnte man verfügbare Charaktere und Monster aus einer Liste auswählen und auf dem Spielfeld platzieren.

Langfristig sollen bessere Bots hinzugefügt werden, die vollständige Encounter automatisch spielen können. Viele Wiederholungen desselben Encounters könnten dann Siegquoten, verbleibende Lebenspunkte und weitere verbrauchte Spielerressourcen erfassen. Damit würde das Projekt wieder an seine ursprüngliche Motivation anschließen: Ein Spielleiter könnte einen vorbereiteten Kampf nicht nur einmal ausprobieren, sondern seine Schwierigkeit anhand mehrerer Simulationen einschätzen.

Außerdem könnte mithilfe der Engine KI-Modelle trainiert werden, die in der Lage sind, einzelne oder sogar mehrere Arten von Charakteren effektiv zu spielen.

6 Literatur- und Quellenverzeichnis

- System Reference Document 5.2.1 inkl. CC-BY-4.0-Lizenz
- Dateien von 5e.tools

7 Anhang

Der Anhang enthält die Schritte zum Starten und Bedienen des Programms sowie ausgewählte Ausschnitte aus den Inhaltsdateien. Die Ausschnitte dienen als Beispiel für das beschriebene Capability-Schema. Die vollständigen Dateien und der übrige Quellcode sind über das Repository verfügbar.

7.1 Bedienungsanleitung

Vorausgesetzt werden Python 3.14 oder neuer und uv. Nach dem Klonen des Repositorys installiert uv sync die in der pyproject.toml festgelegten Abhängigkeiten. Mit uv run srd-arena wird die Anwendung gestartet. Im ersten Fenster wählt der Nutzer einen Encounter aus. Während des Kampfes zeigt die Seitenleiste die für den aktiven Teilnehmer verfügbaren Aktionen. Ziele oder Zielgebiete werden anschließend auf dem Spielfeld ausgewählt und bestätigt. Nach dem Ende des Encounters kann derselbe Kampf neu gestartet oder die Anwendung beendet werden.

7.2 Ausgewählte Code-Snippets

Die folgenden Ausschnitte vergleichen die Capabilities von Burning Hands und Cone of Cold. Beide Zauber verwenden einen vom Zaubernden ausgehenden Kegel, verlangen einen Rettungswurf und verursachen bei einem erfolgreichen Wurf halben Schaden. Unterschiede wie Kegellänge, Rettungswurfsattribut, Schadenswürfel, Schadensart und Skalierung werden durch die Inhaltsdaten beschrieben. Dadurch verwenden beide Definitionen denselben Builder und dieselbe Auflösungslogik.

Cone of Cold:

"You unleash a blast of cold air. Each creature in a 60-foot Cone originating from you makes a
↔ Constitution saving throw, taking 8d8 Cold damage on a failed save or half as much damage on a
↔ successful one. [...]"

...

```
"capability":
  "target":
    "type": "area",
    "origin": "self",
    "geometry":
      "shape": "cone",
      "length_feet": 60
    ,
    "resolution":
      "type": "saving_throw",
      "ability": "con",
      "failure":
        "effects": [
          "type": "damage",
          "dice": "8d8",
          "damage_type": "cold"
        ]
      ,
      "success_damage": "half"
    ,
    "scaling": [
      "type": "slot_level",
      "above_level": 5,
      "per_level": [
        "type": "damage_dice",
        "amount": "1d8"
      ]
    ]
  ]
```

Burning Hands:

"A thin sheet of flames shoots forth from you. Each creature in a 15-foot Cone makes a Dexterity
↔ saving throw, taking 3d6 Fire damage on a failed save or half as much damage on a successful one."

```
"capability":
  "target":
    "type": "area",
    "origin": "self",
    "geometry":
      "shape": "cone",
      "length_feet": 15
    ,
    "resolution":
      "type": "saving_throw",
      "ability": "dex",
      "failure":
        "effects": [
          "type": "damage",
          "dice": "3d6",
          "damage_type": "fire"
        ]
      ,
      "success_damage": "half"
    ,
    "scaling": [
      "type": "slot_level",
      "above_level": 1,
      "per_level": [
        "type": "damage_dice",
        "amount": "1d6"
      ]
    ]
  ]
```